

.NET Runtime Performance & Architecture

Technical Summary Report on JIT Optimization and Core .NET Concepts

Section 1: Overhead of JIT and Ways to Reduce It

JIT compilation is the transformation of IL-code into the native code in runtime. However, despite the possibility of the platform independence and dynamic optimization of code, JIT brings some overhead to start-up times and memory consumption.

Tiered Compilation: The current .NET runtime works with the tiered compilation. At the first run of an application, the runtime uses the fast and non-optimizing compiler (Tier 0). After that, in the background, the most often-called methods are recompiled by the optimized compiler (Tier 1).

Ahead-Of-Time (AOT) Compilation: Technologies like ReadyToRun (R2R) and Native AOT provide the compiling of IL-code into the native code before the execution without the overhead of JIT.

Profile-Guided Optimization (PGO): In the PGO the runtime collects the information about the program execution, optimizes code paths, inlines the methods and performs efficient memory layout, thus decreasing the overhead of JIT.

Assembly Trimming & Pre-JITing: In the Native AOT and **NGen** (old .NET Framework technology) pre-compilation of assemblies happens before deployment. Thus, there is no overhead from the JIT compilation.

Section 2: Basic Concepts of .NET

1. .NET versions

- **.NET Framework:** Legacy Windows-only platform (versions 1.0 to 4.8.1).
- **.NET Core:** Cross-platform and high-performance redesign of .NET Framework. The open-source and free platform (versions 1.0 to 3.1).
- **Unified .NET (.NET 5+):** Unification of .NET Core and .NET Framework into the one cross-platform development platform (versions .NET 5, 6, 7, 8, 9).

2. .NET Core architecture

- Design for cloud-native applications and microservices with cross-platform deployments (Windows, Linux, macOS).
- Rich feature set provided with the help of modular ecosystem and using of lightweight NuGet packages.

3. Namespaces

- **Purpose:** Mechanism of logical grouping and separation of types (classes, interfaces, structs, enums) from other libraries and preventing of the name collisions.
- **Usage:** Defined in C# by `namespace MyNamespace { ... }` and used by the `using` directive.

4. Solutions (`.sln`)

- Container in Visual Studio/.NET CLI for one or more `.csproj` projects.
- Manages the build configuration and the relations between projects.